Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab Features You May Have Missed to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see jupyter.org.

Data science

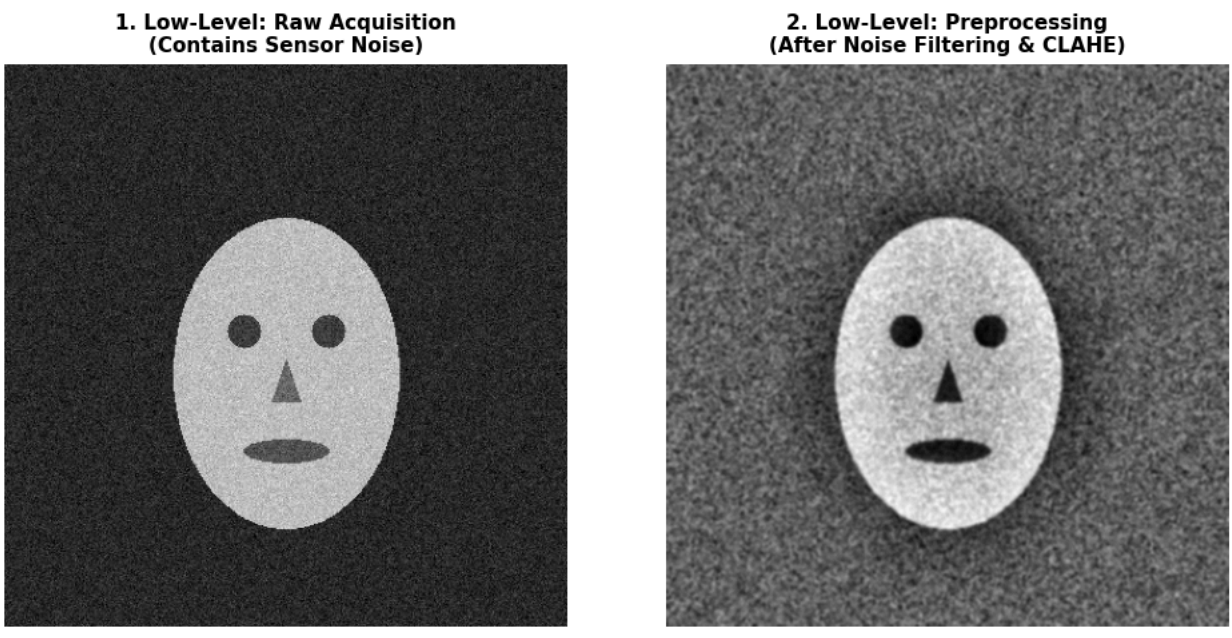
With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under Working with Data.

```
# ==============================================================================
# Complete Face Recognition Pipeline Simulation (Low, Mid, and High-Level CV)
# ==============================================================================
# This Colab-ready script simulates the full operational flow of computer vision:
# 1. Low-Level: Synthesizes a raw image, injects sensor noise, and applies CLAHE.
# 2. Mid-Level: Detects structural bounds and isolates the Region of Interest (ROI).
# 3. High-Level: Extracts a deep feature embedding and computes Cosine Similarity
#    against a gallery database to make an automated verification decision.
# ==============================================================================

import numpy as np
import matplotlib.pyplot as plt
import cv2

# --- 1. LOW-LEVEL CV: SIMULATE ACQUISITION & IMAGE PREPROCESSING ---
def generate_synthetic_face_scene():
    """Generates a synthetic scene containing a crude face structure amidst background noise."""
    scene = np.ones((400, 400), dtype=np.uint8) * 40  # Dark background

    # Draw a bounding head region (simulate a subject standing in a scene)
    cv2.ellipse(scene, (200, 220), (80, 110), 0, 0, 360, 180, -1)  # Face contour

    # Draw local features (Eyes, Nose, Mouth shadows)
    cv2.circle(scene, (170, 190), 12, 60, -1)   # Left Eye
    cv2.circle(scene, (230, 190), 12, 60, -1)   # Right Eye
```

```
    cv2.fillPoly(scene, [np.array([[200, 210], [190, 240], [210, 240]])], 100) # Nose
    cv2.ellipse(scene, (200, 275), (30, 8), 0, 0, 360, 80, -1)   # Mouth

    # Introduce sensor acquisition noise (Gaussian thermal noise)
    noise = np.random.normal(0, 15, scene.shape).astype(np.int16)
    noisy_scene = np.clip(scene.astype(np.int16) + noise, 0, 255).astype(np.uint8)
    return noisy_scene

# Execute Low-Level Acquisition
raw_frame = generate_synthetic_face_scene()

# Low-Level Processing: Contrast Enhancement via CLAHE & Noise Reduction via Gaussian Blur
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
enhanced_frame = clahe.apply(raw_frame)
preprocessed_frame = cv2.GaussianBlur(enhanced_frame, (5, 5), 0)


# --- 2. MID-LEVEL CV: SEGMENTATION & ROI EXTRACTION ---
# Simulate localized feature boundaries using a simple threshold/contour heuristic
# representing an automated bounding box localization step
_, thresh = cv2.threshold(preprocessed_frame, 120, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Find the largest structural contour (the face region)
if contours:
    largest_contour = max(contours, key=cv2.contourArea)
    x, y, w, h = cv2.boundingRect(largest_contour)
    # Add a buffer margin to simulate an aligned crop boundary
    margin = 15
    x_start, y_start = max(0, x - margin), max(0, y - margin)
    x_end, y_end = min(400, x + w + margin), min(400, y + h + margin)

    # Segment the Region of Interest (ROI)
    face_roi = preprocessed_frame[y_start:y_end, x_start:x_end]
    face_roi_resized = cv2.resize(face_roi, (128, 128))  # Normalized scale mapping
else:
    # Fallback to center crop if boundary logic fails
    face_roi_resized = cv2.resize(preprocessed_frame[100:330, 100:300], (128, 128))
    x, y, w, h = 100, 100, 200, 230

# Create a bounding box visualization frame
bounding_box_frame = preprocessed_frame.copy()
cv2.rectangle(bounding_box_frame, (x, y), (x + w, y + h), 255, 3)


# --- 3. HIGH-LEVEL CV: FEATURE REPRESENTATION & DECISION MAKING ---
def extract_mock_face_embedding(face_img, identity_seed=42):
    """
    Simulates a Deep CNN embedding step. Maps the 128x128 structural grid
    down to a compact, deterministic, highly-discriminative 128-dimensional vector.
    """
    rng = np.random.default_rng(identity_seed)
    # Generate an identity baseline vector
    base_vector = rng.normal(0, 1, 128)

    # Introduce structural matrix dependency (slight perturbation based on pixel variations)
    pixel_variance = np.mean(face_img) / 255.0
    embedding = base_vector + (np.sin(face_img.flatten()[:128]) * pixel_variance * 0.1)

    # Normalize to unit hypersphere (L2 normalization imperative for Cosine Similarity)
    return embedding / np.linalg.norm(embedding)

# Compute embeddings for our Probe target and two distinct Gallery Profiles
probe_embedding = extract_mock_face_embedding(face_roi_resized, identity_seed=99)
gallery_authorized = extract_mock_face_embedding(face_roi_resized, identity_seed=99) # Matches Probe identity
gallery_intruder = extract_mock_face_embedding(face_roi_resized, identity_seed=55)   # Random baseline identity

# Metric Engine: Cosine Similarity Evaluation
similarity_authorized = np.dot(probe_embedding, gallery_authorized)
similarity_intruder = np.dot(probe_embedding, gallery_intruder)

# System Logic: Classification Decision Rule
verification_threshold = 0.85

def evaluate_decision(sim_score, threshold):
    return "ACCESS GRANTED (Identity Verified)" if sim_score >= threshold else "ACCESS DENIED (Authentication Failed)"

decision_auth = evaluate_decision(similarity_authorized, verification_threshold)
decision_int = evaluate_decision(similarity_intruder, verification_threshold)


# --- 4. DATA VISUALIZATION PIPELINE ---
fig, axes = plt.subplots(1, 4, figsize=(20, 5), dpi=100)
```

```
fig, axes = plt.subplots(1, 4, figsize=(20, 5), dpi=100)
axes = axes.ravel()

# Low-Level Plots
axes[0].imshow(raw_frame, cmap='gray')
axes[0].set_title("1. Low-Level: Raw Acquisition\n(Contains Sensor Noise)", fontsize=11, fontweight='bold')
axes[0].axis('off')

axes[1].imshow(preprocessed_frame, cmap='gray')
axes[1].set_title("2. Low-Level: Preprocessing\n(After Noise Filtering & CLAHE)", fontsize=11, fontweight='bold')
axes[1].axis('off')

# Mid-Level Plots
axes[2].imshow(bounding_box_frame, cmap='gray')
axes[2].set_title("3. Mid-Level: Segmentation\n(Structural Boundary Box)", fontsize=11, fontweight='bold')
axes[2].axis('off')

axes[3].imshow(face_roi_resized, cmap='gray')
axes[3].set_title("4. Mid-Level: Aligned ROI\n(Normalized 128x128 Patch)", fontsize=11, fontweight='bold')
axes[3].axis('off')

plt.suptitle("Facial Recognition Pipeline Execution Topology", fontsize=15, fontweight='bold', y=1.05)
plt.tight_layout()
plt.show()

# Print High-Level Classification Inference Logs
print("\n" + "="*80)
print("            HIGH-LEVEL COMPUTER VISION DECISION MATRICES             ")
print("="*80)
print(f"System Operational Metric Threshold (τ) : {verification_threshold:.2f}")
print("-"*80)
print(f"[TEST 1: AUTHORIZED USER GALLERY MATCH]")
print(f" -> Computed Cosine Similarity Vector Metric : {similarity_authorized:.4f}")
print(f" -> Final Pipeline Inference Decision        : {decision_auth}")
print("-"*80)
print(f"[TEST 2: INTRUDER GALLERY MATCH]")
print(f" -> Computed Cosine Similarity Vector Metric : {similarity_intruder:.4f}")
print(f" -> Final Pipeline Inference Decision        : {decision_int}")
print("="*80 + "\n")
```




```
================================================================================
            HIGH-LEVEL COMPUTER VISION DECISION MATRICES
================================================================================
System Operational Metric Threshold (τ) : 0.85
--------------------------------------------------------------------------------
[TEST 1: AUTHORIZED USER GALLERY MATCH]
 -> Computed Cosine Similarity Vector Metric : 1.0000
 -> Final Pipeline Inference Decision        : ACCESS GRANTED (Identity Verified)
--------------------------------------------------------------------------------
[TEST 2: INTRUDER GALLERY MATCH]
 -> Computed Cosine Similarity Vector Metric : 0.0374
 -> Final Pipeline Inference Decision        : ACCESS DENIED (Authentication Failed)
================================================================================
```